

Objektum orientált paradigma és annak megvalósítása a JAVA és C++ nyelvekben. Az absztrakt adattípus, az osztály. Az egységbe zárás, az információ elrejtés, az öröklődés, az újrafelhasználás és a polimorfizmus. A polimorfizmus feloldásának módszere.

OBJEKTUM ORIENTÁLT PARADIGMA ÉS ANNAK MEGVALÓSÍTÁSA JAVA ÉS C++ NYELVEKBEN

OOP alapfogalmak

- olyan szemléleti mód (paradigma), mely a szoftverfejlesztést modellezésként fogja fel, tehát nem utasítások listájaként, hanem egymással kommunikáló önálló egységekként (objektumok) kezeli
- **Absztrakt Adattípusok**
 - egy olyan matematikai modell, amely az adatokat és a rajtuk végezhető műveletek összességét egyetlen logikai egységbe zárja
 - két részből állhat:
 - **értékkészlet:** milyen értékeket vehetnek fel az adatok
 - **műveletekhalmaza:** az adatokon milyen műveletek végezhetők el
 - lényege, hogy elrejtí a megvalósítást, azaz a felhasználónak elég azt tudni mit csinál a típus (műveletek listája, bemenetek és kimenetek), azt, hogy hogyan (implementációs rész) az neki nem számít
- **Osztályok**
 - egy **absztrakt adattípus**, a felhasználó által definiált adat, ami egy tervrajzként / sablonként működik
 - két fő komponense van
 - **attribútumok (mezők):** objektum belső állapotai
 - **metódusok (tagfüggvények):** az objektumok viselkedését definiálják
 - tehát egy osztály leírja azonos attribútumokkal és metódusokkal rendelkező objektumok egy csoportját
 - C++ és JAVA esetén is kulcsszava a **class**
 - JAVA esetén minden osztály az **Object** osztályból öröklődik
- **Objektumok (példány)**
 - az objektumok az osztályok példányai
 - objektum az osztály alapján a memóriában létrejött konkrét entitás
 - minden objektum rendelkezik 3 dologgal:
 - **állapot:** attribútumok, mezők aktuális értékei
 - **viselkedés:** metódusok, azaz, hogy reagál bizonyos kérésekre
 - **identitás:** minden objektum egyedi, azaz megkülönböztethető más objektumoktól (memóriacím vagy referencia alapján)
 - még ha állapotai egyeznek is két objektumnak azok akkor is különbözőnek számítanak
- az objektumok **metódusok** hívásával kommunikálnak egymással
- 4 alappillére épül: **egységbezárás, absztrakció, öröklődés és polimorfizmus**

Egységbezárás (encapsulation)

- egységbezárás azt jelenti, hogy az adatokat (attribútumokat) és a rajtuk végzett műveleteket (metódusokat) egy logikai egységbe (osztály) kezeljük
- fő célja az **információ elrejtés**
 - az objektum belső állapota nem érhető el közvetlenül a külvilág számára csak vezérelt módon, publikus metódusokon keresztül
 - publikus getter/setter és private attribútumok
 - az információ elrejtés biztosítja az integritást
 - pl: nem lehet érvénytelen adatot beállítani (pl: negatív életkor), mert a setter ellenőrizheti a bemenetet
 - ezt a **láthatósági (hozzáférési) módosítókkal** érjük el
 - **private**: csak az osztályon belül látható
 - ajánlott szintje az adatoknak
 - **default** (package-private) - CSAK JAVA-ában
 - csak az azonos csomagban (package) lévő osztályok látják
 - **protected**: a csomagon belül és az osztályon belül, illetve a gyermekosztályok számára látható (akár más csomagban lévő gyermekosztályok számára is)
 - **public**: bárki számára elérhető
 - ajánlott szintje a metódusoknak
 - C++ esetén szekciók vannak (public: ...)
 - ilyenkor ezután minden az lesz a következő módosítóig
 - JAVA esetén a láthatósági módosítókat minden tag elé ki kell írni
 - C++-ban a class alapértelmezett láthatósága private, a struct pedig public
 - C++ esetén van egy *friend* kulcsszó, ezzel egy osztály egy másik osztályt vagy függvényt tud a barátjává fogadni, ilyenkor a barát látja az osztály private tagjait is
 - hatékonyság miatt hasznos, de óvatosan kell használni, mert gyengíti az egységbezárást (információ elrejtést)
 - pl: operátor overloadnál lehet alkalmazni
- előnye, hogy ha a belső megvalósítás meg is változik akkor is a külvilág felé mutatott interfész változatlan maradhat és a többi programrészben nem kell módosításokat végezni

Absztrakció

- célja, hogy a felesleges dolgok, mint a megvalósítás eltűnjön
 - azaz segít fókuszálni arra, hogy mit csinál egy entitás, ahelyett, hogy arra fókuszálna, hogy hogyan csinálja azt
- az **interfészek** és **absztrakt osztályok** valósítják ezt meg
 - az **interfész** (csak JAVA) segítségével egy osztály váza írható le, azaz, hogy milyen metódusokkal rendelkezik, anélkül, hogy ezek implementációra kerülnének
 - metódusokat definiálnak, amiket más osztályok implementálhatnak (**implements** kulcsszóval)
 - ha az adott osztályokba legalább 1 metódusa az interfésznek nem kerül implementálásra, akkor az osztály absztrakt lesz
 - ```
interface MyInterface {
 final int a = 10;
 public void display();
}

class MyClass implements MyInterface {
 public void display() { /*implementation*/ }
}
```
    - az interfész is egy absztrakt adattípus
      - attribútumai alapból: public, final és static
        - nem tartalmazhat interfész példányváltozókat, mert alapból statikusok az attribútumok
      - a metódusok alapból: public és abstract
        - final nem lehet, mert a final metódusok nem írhatók felül
        - JAVA 9 óta lehet private metódus
      - az interfész nem tartalmaz konstruktort
    - JAVA esetén a többszörös öröklődés interfészekkel megoldható
      - egy osztály több interfészt is implementálhat
      - JAVA 8 és a fölött, mert **default metódusok** szükségesek
        - default metódus = lehet a metódusnak törzse
        - ha egy osztály 2 interfészt valósít meg és ezeknek van azonos nevű default metódusa akkor az **@Override** szükséges
    - C++-ban nincs interface, de absztrakt osztállyal amibe minden metódus teljesen virtuális (virtual void f() = 0;) megoldható ugyanolyan viselkedés mint egy JAVA interfésszel
  - az **absztrakt osztályok** olyan osztályok, amik nem példányosíthatók, és tartalmazhatnak absztrakt metódusokat (kód nélküli deklarációkat)
    - JAVA-ban (abstract kulcsszó) és C++-ban is vannak (olyan osztály aminek van egy teljesen virtuális metódusa (virtual void f() = 0;))
    - amennyiben egy osztálynak van 1 absztrakt metódusa akkor az osztály automatikusan absztrakt lesz
    - céljuk, hogy ősként szolgáljanak a származtatáshoz, kikényszerítve bizonyos metódusok megvalósítását a leszármazottakban
    - többszörös öröklődésre JAVA esetén nem használhatók

## Öröklődés

- az öröklődés lehetővé teszi, hogy meglévő osztályokból (ősosztály, szülő) új osztályokat (származtatott osztály, gyerek) hozzunk létre
  - a gyerekosztály örökli a szülő tulajdonságait (attribútumok) és viselkedését (metódusok), ezeket kiegészítheti vagy módosíthatja is
    - JAVA és C++ esetén is csak a nem private metódusokat örökli az őszosztálytól
    - alpból a konstruktort nem örökölhetheti egyiknél sem, de C++-ban (11 verziótól using kulcsszóval megoldható már a konstruktor öröklése is)
  - az örökölt dolgok megvalósítását felülírhatjuk (polimorfizmus), illetve a láthatóságait is módosíthatjuk de csak szigorítási formába (ha public volt akkor lehet protected és private is, de ha private volt akkor se public, se protected)
    - JAVA esetén konstruktor ha private akkor public-á módosítható a gyerekosztályba
  - JAVA-ban és C++-ban nincs **attribútum felülírás** (field overloading)
- célja a **kód újrafelhasználása** és a **hierarchikus rendezés**
- **Megvalósítás**
  - JAVA: class DogClass **extends** AnimalClass
  - C++: class DogClass: public AnimalClass
    - öröklődés láthatósága is szabályozható
      - public öröklődés
        - leggyakorabbi, megtartja az eredeti láthatóságokat
      - protected vagy private öröklődés
        - ritkább és az ős publikus tagjait korlátozza az öröklődés láthatóságára
          - protected esetén a publikus tagok protecteddé
          - private esetén a public és protected tagok private-té
  - többszörös öröklődés osztályokkal:
    - JAVA esetén csak egyszeres öröklődés működik (osztályok esetén)
    - C++ esetén a többszörös öröklődés is támogatott
      - class DogClass: public AnimalClass, public LivingClass
  - JAVA final és C++11-es verziójától a final kulcsszóval ellátott osztályból nem lehet tovább örökölni
  - **super**
    - az őszosztály metódusait, konstruktorát, vagy attribútumait a super kulcsszóval érjük el a gyermek osztályba
      - super.function1(), super() - constructor, super.attribute1
    - a gyerekosztályba az őse konstruktorát meg kell hívni hierarchikus sorrendben
      - tehát a gyerek konstruktorba először a super() kerül meghívásra

- **újrafelhasználás egyéb formái**
  - **kompozíció (összetétel - erős kapcsolat)**
    - az alkatrész élettartama az egész objektum élettartamától függ, azaz ha az egész megsemmisül, az alkatrész is
    - pl: egy Ház osztály és annak Szoba objektumai, ha lebontják a házat, a szobák is megszűnnek
    - C++-ban ezt leggyakrabban érték szerinti adattaggal valósítjuk meg (nem mutatóval).
  - **aggregáció (csoportosítás - gyenge kapcsolat)**
    - az alkatrész (a tartalmazott objektum) független az egésztől, létezhet nélküle is
    - pl: egy EgyetemiTanszék és az ott dolgozó Oktatók, ha a tanszék megszűnik, az oktatók még léteznek (elmennek máshova dolgozni)
    - C++-ban ezt mutatókkal vagy referenciákkal valósítjuk meg
- **gyémánt öröklődés**
  - JAVA esetén nincs többszörös öröklődés osztályokkal, így nem jöhet elő, C++-ban viszont igen
  - lényege, hogy van egy ősosztály (Jármű) és van két gyermekosztálya (Vizijármű és Légijármű), és van egy másik osztály (Hidroplane) ami ebből a két gyermekosztályból öröklődik
    - két problémája van
    - **kétértelműség**, ha a származtatott osztály meghív egy metódust, amelyet az ősosztály definiált, és az A, illetve B osztály is felülírta (vagy nem), a fordító nem tudja eldönteni, hogy melyik ágon érkező metódust kell futtatnia
    - **duplikáció**, az ősosztály adattagai mindkét gyermekosztályból bekerülnek, ez memória pazarlás
  - C++-ban a megoldás a virtuális öröklődés:

```
class Jarmu
class Vizijarmu: public virtual Jarmu
class Legijarmu: public virtual Jarmu
class Hidroplane: public Vizijarmu, public Legijarmu
```

    - virtuális öröklődésnél a legalsó osztály (Hidroplane) kötelessége a fő ősosztály konstruktorának meghívása, így csak 1x fut le

## Polimorfizmus

- azt jelenti, hogy különböző típusú objektumokat (pl. Kutya, Macska) közös ősük (pl. Állat) típusán keresztül tudunk kezelni, és az objektumok a saját típusuknak megfelelően fognak viselkedni
- segít, hogy a kód flexibilis és újrahasználatos legyen
- JAVA-ban minden objektumváltozó valójában egy referencia (mutató)
  - amikor azt írod: `Allat a = new Kutya();`, a háttérben egy mutatót kezelsz, ezért működik "automatikusan" a polimorfizmus

- C++ esetén ha azt írod: Allat a;, a fordító pontosan annyi helyet foglal a memóriában, amennyi egy Allat-nak kell
  - ha ebbe belepróbálunk gyömöszölni egy Kutya objektumot, az nem fog beleférni
- **Statikus polimorfizmus (korai hozzárendelés)**
  - **metódus túlterhelés (overload)**
    - ugyanazon osztályon belül több metódusnak is ugyanaz a neve, de a paraméterlistájuk (típus vagy számosság) különböző
    - pl: szamol(int a, int b) és szamol(double a, double b).
  - fordító már **fordítási időben** tudja melyik metódus kerül meghívásra, a paraméterek alapján
- **Dinamikus polimorfizmus (késői hozzárendelés)**
  - **metódus felülírás (override)**
    - gyerekosztály felülírhatja az őszosztályának metódusait
    - itt a paraméterlista nem változik
  - lehetővé teszi, hogy a különböző osztályok eltérő viselkedését ugyanazon az interfészen keresztül használjuk
    - példa
      - Allat őszosztály hangotAd() metódusát, a Kutya felülírja (vau), a Macska felülírja (miau)
      - *Kód logikája*
        - létrehozol egy listát, amiben Allat referenciák vannak
        - belepakolsz egy kutyát és egy macskát, majd végigmész a listán és mindegyiknek kiadod a parancsot: hangotAd()
      - *Eredmény:* Bár a listában "csak" állatokként látod őket, a kutya ugatni fog, a macska nyávogni, ez a polimorfizmus: **nem kell tudnod, pontosan mi az, csak azt, hogy egy Állat, és tud hangot adni.**
  - **futási időben** történik (JVM kezeli), híváskor dől el pontosan melyik metódus kerül meghívásra
    - nem a fordító dönt arról melyik metódus fusson le
  - **C++ nyelvben a dinamikus polimorfizmus**
    - C++-ban alapértelmezésben a kötés statikus
      - ha egy őszosztály típusú mutatóval hivatkozunk egy gyerekobjektumra, és meghívunk egy metódust, az őszosztály metódusa fut le
    - a dinamikus viselkedéshez a *virtual* kulcsszót kell használni
      - ha egy metódusról tudjuk, hogy azt a leszármazott osztályokban felül szeretnénk írni, akkor az legyen virtual

- a háttérben a VMT (Virtual Method Table) segítségével valósul meg:
  - minden olyan osztályhoz, amelynek van virtuális függvénye, a fordító létrehoz egy statikus táblázatot (VMT)
    - ez tartalmazza az osztály virtuális metódusainak memóriacímeit (függvénymutatókat)
  - ha egy gyerekosztály felülír (override) egy metódust, az ő VMT-jében az új kód címe fog szerepelni
  - minden objektumpéldány kap egy rejtett adattagot (**vptr**, azaz Virtual Pointer) a konstruktor lefutásakor, mely az adott objektum osztályának VMT-jére mutat
- **hívás folyamata (feloldás)**, az az mikor egy virtuális metódust hívunk (allat->hangotAd()):
  2. megkeresi az objektum vptr-ét
  3. vptr alapján eljut az osztály VMT-jéhez
  4. a táblázatból kikeresi a megfelelő metódushoz tartozó memóriacímet
  5. majd erre a címre ugrik a vezérlés (végrehatja a kódot)
- ez a mechanizmus kis teljesítményveszteséggel jár (indirekció), de ez az ára a rugalmasságnak
  - lépcsős memóriahozzáférés (mutató követése -> táblában keresés -> ugrás)
- emellett az *override* kulcsszó kiírása is fontos segéd lehet, nem kötelező, de segít abba ha elírnánk a nevét vagy paraméter listáját a felülírandó függvénynek (azaz igazából elírás miatt végül nem is íránk felül semmit)

```
class Allat{
 public:
 virtual int szamol(int kor){/**/}
 void kiir(){/**/}
};
class Kutya: public Allat{
 public:
 int szamol() override{/**/} //Hibat fog dobni, mivel nincs olyan metodus amit foleirna (parameterlista miatt)
 int szamol(int kor) override{/**/} //OK
 void kiir() override {/**/} //Hibat fog dobni, mivel a kiir az nem egy virtualis metodus
};
```

- **JAVA nyelvben a dinamikus polimorfizmus**
  - interfész, absztrakt osztályok és late binding segítségével
  - JAVA esetén a metódusok alapértelmezetten virtuálisak (kivéve, static, final és private), így a dinamikus kötés (late binding) automatikus
    - late binding = gyerekosztályra úgy hivatkozunk mint az ősoosztály
    - JAVA-ban List<String> felcserélhető vele ArrayList vagy LinkedList is

- **Polimorfizmus feloldása**

- típus kasztolással történik
  - `int i = (int) i;`
  - `int i = static_cast<int>(d);`
- pl: Allat a; változó hivatkozhat egy speciális objektumra, mint például a Kutya. ekkor néha szükséges az eredeti típus visszaállítása
- **Upcasting** (gyerek -> szülő), ez automatikus és biztonságos
- **Downcasting** (szülő -> gyerek), ezt explicit módon kell jelezni
  - `Allat a = new Kutya();`  
`Kutya k = (Kutya) a;`
  - veszélyes lehet akkor ha az objektum valójában nem az amire kasztoljuk (imménti példában ha igazából nem Kutya)
    - ezért fontos a **típusellenőrzés**
      - JAVA-ban: `instanceof` operátor
      - C++-ban: `dynamic_cast<Kutya>` vagy `is_base_of`

```
subpoint s;

point & a = s;
subpoint & b1 = static_cast<subpoint&>(a);
subpoint & b2 = dynamic_cast<subpoint&>(a);
```

The first ( `static_cast` ) is more dangerous; there is no check that the conversion is valid, so if `a` doesn't refer to a `subpoint` , then using `b1` will have undefined behaviour.

The second ( `dynamic_cast` ) is safer, but will only work if `point` is polymorphic (that is, if it has a virtual function). If `a` refers to an object of incompatible type, then it will throw an exception.